CS527 – Computer Systems
Lab: Introduction to OpenMP — Arrays and Matrices

General Instructions:
- Please check Part-II of the ``Parallel Programming in MPI and OpenMP'' book. Chapter 17 onwards. The book has been shared in GC already.
- You can work as buddies, but submission has to be done individually.
- Language: C or C++.
- Compile with OpenMP enabled:
  ``` *gcc -fopenmp -O2 task1_vecadd.c -o task1* ```
- For every task, implement **both** a serial version and an OpenMP-parallel version of the same function, and verify their outputs match before reporting timing.
- Use `*omp_get_wtime()*` to measure execution time (not `clock()`), e.g.:
  ```c
  *double start = omp_get_wtime();*
  *// ... work ...*
  *double end = omp_get_wtime();*
  *printf("Time: %f seconds\n", end - start);*
  ```
- Set the number of threads either via `*export OMP_NUM_THREADS=4*` before running, or with `*omp_set_num_threads(n)*` in code. Report which method you used.
- For each task, submit: source code, a short `README`/report with your timing table, and a 2–3 line explanation of the observed speedup (or lack of it).

Task 1 — Vector Addition

Goal: Get comfortable with the basic `parallel for` construct.
Problem statement:
Given two arrays `a[N]` and `b[N]` of `double`s, compute `c[i] = a[i] + b[i]` for all `i`, where `N` is at least 10,000,000.
Requirements:
1. Write a serial loop that computes `c`.
2. Write a parallel version using `*#pragma omp parallel for*`.
3. Verify the two results are identical (compare element-wise, allow no floating-point tolerance since addition here is exact for the values you choose).
4. Measure and report execution time for `N = 10^5, 10^6, 10^7, 10^8` and for thread counts `1, 2, 4, 8` (or however many cores your machine has).
5. Plot or tabulate speedup = `T_serial / T_parallel` against thread count for the largest `N`.

Questions to address in your report:
- At what array size does parallelization actually start paying off? Why is it not beneficial for small `N`?
- Does speedup keep increasing linearly with thread count? Why or why not?

Sample code: using C

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
```
```

```
void vec_add_serial(double *a, double *b, double *c,
long n) {
    for (long i = 0; i < n; i++)
        c[i] = a[i] + b[i];
}

void vec_add_parallel(double *a, double *b, double *c,
long n) {
    #pragma omp parallel for
    for (long i = 0; i < n; i++)
        c[i] = a[i] + b[i];
}

int main(int argc, char *argv[]) {
    long n = /* read from argv or hardcode */;
    double *a = malloc(n * sizeof(double));
    double *b = malloc(n * sizeof(double));
    double *c_serial = malloc(n * sizeof(double));
    double *c_parallel = malloc(n * sizeof(double));

    // TODO: initialize a[], b[] with some values

    double t0 = omp_get_wtime();
    vec_add_serial(a, b, c_serial, n);
    double t1 = omp_get_wtime();
    vec_add_parallel(a, b, c_parallel, n);
    double t2 = omp_get_wtime();

    // TODO: verify c_serial == c_parallel
    // TODO: print timings

    return 0;
}
```

Task 2 — Sum of Array Elements

Goal: Learn the `reduction` clause and understand why a naive parallel sum is unsafe.
Problem statement: Given an array `a[N]` of integers (or doubles), compute the total sum of all elements.

Requirements:
1. First, write a parallel version **without** any reduction clause or synchronization(i.e., every thread directly does `sum += a[i]`) and run it. Report what goes wrong — run it multiple times and show that the result is inconsistent/incorrect.
2. Fix it using `*#pragma omp parallel for reduction(+:sum)*`.
3. Verify the reduction-based result matches a serial sum, every run.
4. Measure timing for `N = 10^7, 10^8` across thread counts `1, 2, 4, 8`.

Questions to answer in your report:
- Why does the naive version (step 1) give wrong/inconsistent answers? Explain in terms of a data race on the shared variable `sum`.
- Would using `*#pragma omp critical*` around `*sum += a[i]*` fix correctness? What would you expect to happen to performance if you did that instead of using `reduction`? (You may implement this and measure it.)

Sample code: using C

```
long sum_serial(long *a, long n) {
    long sum = 0;
    for (long i = 0; i < n; i++)
        sum += a[i];
    return sum;
}

long sum_naive_parallel_BROKEN(long *a, long n) {
    long sum = 0;
    #pragma omp parallel for
    for (long i = 0; i < n; i++)
        sum += a[i];      // <-- data race,
intentionally left broken for you to observe
    return sum;
}

long sum_reduction(long *a, long n) {
    long sum = 0;
    #pragma omp parallel for reduction(+:sum)
    for (long i = 0; i < n; i++)
        sum += a[i];
    return sum;
}
```

Task 3 — Maximum and Minimum of an Array

Goal: Extend the reduction idea to `max`/`min`, and practice finding not just the value but also its index.

Problem statement:
Given an array `a[N]` of integers, find:
(a) the maximum value, and
(b) the index at which that maximum occurs.

Requirements:
1. Compute the maximum value using `*#pragma omp parallel for reduction(max:max_val)*`.
(If your compiler's OpenMP version doesn't support `max`/`min` reductions — check with `gcc --version`, OpenMP 4.0+ is required — implement a manual reduction using a per-thread local max and combine at the end, or use `critical`.)
2. Separately, find the **index** of the maximum. This is harder because you're combining two pieces of information (value and index) — a plain `reduction(max:)` on the value alone won't give you the index. Implement this using either:
   - a `critical` section that updates a shared `(best_val, best_idx)` pair, or
   - a manual reduction: each thread finds its own local `(val, idx)` over its chunk
     (using `*omp_get_thread_num()*` and manual loop bounds, or an array of per-thread results), then a final serial pass picks the overall best.
3. Repeat for minimum.
4. Verify against a serial implementation.
5. Measure timing for `N = 10^7, 10^8`.

Questions to answer in your report:

- Why can't you directly write `reduction(max: best_idx)` to get the index of the maximum?
- Compare the performance of the `critical`-section approach vs. the manual per-thread reduction approach for finding `(max_val, max_idx)`. Which is faster, and why?

Sample(critical-section approach) using C:

```
void find_max_idx_critical(int *a, long n, int *max_val, long *max_idx) {
    int best_val = a[0];
    long best_idx = 0;

    #pragma omp parallel for
    for (long i = 1; i < n; i++) {
        #pragma omp critical
        {
            if (a[i] > best_val) {
                best_val = a[i];
                best_idx = i;
            }
        }
    }
    *max_val = best_val;
    *max_idx = best_idx;
}
```

Task 4 — Matrix Addition

Goal: Move from 1D arrays to 2D, and compare parallelizing the outer loop only vs. using `collapse(2)`.
Problem statement:
Given two `N x N` matrices `A` and `B` (stored as 2D arrays or flattened 1D arrays — your choice, but be explicit about which you use), compute `C[i][j] = A[i][j] + B[i][j]`.

Requirements:
1. Implement the serial version with nested loops.
2. Implement a parallel version parallelizing "only the outer loop":
   Using c
   ```
   #pragma omp parallel for
   for (i = 0; i < N; i++)
       for (j = 0; j < N; j++)
           C[i][j] = A[i][j] + B[i][j];
   ```
3. Implement a second parallel version using `collapse(2)`:
   Using C
   ```
   #pragma omp parallel for collapse(2)
   for (i = 0; i < N; i++)
       for (j = 0; j < N; j++)
           C[i][j] = A[i][j] + B[i][j];
   ```
4. Verify both parallel versions match the serial result.
5. Measure timing for at least two matrix sizes: one where `N` is **small but the matrix is "tall"** in some sense (e.g., `N = 8`, so the outer loop has very few iterations to distribute among threads) and one where `N` is large (e.g., `N = 4096`). Use thread counts `1, 2, 4, 8`.

Questions to answer in your report:
- For the small-`N` case, does `collapse(2)` help compared to parallelizing only the outer loop? Explain why, in terms of how many

independent iterations are available to distribute across threads in each version.
- For the large-`N` case, does `collapse(2)` still make a measurable difference, or is parallelizing the outer loop alone already enough to keep all threads busy?

Task 5 — Matrix-Matrix Multiplication (Naive)

Goal: Parallelize the classic triple-nested-loop matrix multiplication and study how choice of parallelized loop and `collapse` affect performance.

Problem statement:
Given two `N x N` matrices `A` and `B`, compute `C = A x B` using the standard triple-loop algorithm:

```
for i in 0..N:
  for j in 0..N:
    for k in 0..N:
      C[i][j] += A[i][k] * B[k][j]
```

Requirements:
1. Implement the serial triple-loop version. Make sure `C` is zero-initialized first.
2. Implement a parallel version that parallelizes "only the outermost (`i`) loop":
```
   #pragma omp parallel for
   for (i = 0; i < N; i++)
       for (j = 0; j < N; j++)
           for (k = 0; k < N; k++)
               C[i][j] += A[i][k] * B[k][j];
```
2. Implement a second version using `collapse(2)` on the `i` and `j` loops (do not collapse the `k` loop — explain in your report why collapsing the reduction loop `k` would be incorrect/unsafe without extra care).
3. Verify both parallel versions produce the same `C` as the serial version (use a tolerance-based comparison if using floating-point matrices).
4. Measure timing for `N = 256, 512, 1024` with thread counts `1, 2, 4, 8`. Report speedup and parallel efficiency (`speedup / num_threads`) for each.
5. **Bonus (optional):** Try swapping the loop order to `i, k, j` instead of `i, j, k` (this changes the memory access pattern on `B` and `C` from column-wise to row-wise).
   Measure whether this loop-order change affects the serial baseline time, and whether it changes how well the parallel version scales.

Questions to answer in your report:
- Why is it safe to parallelize the `i` loop directly (no race condition), even though the innermost `k` loop is doing a summation into `C[i][j]`?
- What happens to correctness if you try to parallelize the `k` loop directly with a plain `parallel for` (no reduction)? Explain the race condition that would occur on `C[i][j]`.
- Compare `parallel for` on `i` alone vs. `collapse(2)` on `(i, j)`. For which values of `N` (relative to your thread count) would you expect `collapse(2)` to matter most?

Sample code segment using C:

```
void matmul_parallel(double **A, double **B, double **C, int n) {
    #pragma omp parallel for collapse(2)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
```

(Note: accumulating into a local `sum` before writing `C[i][j]` once, rather than repeatedly writing `C[i][j] += ...` inside the `k` loop, avoids redundant memory traffic and is generally the better style — think about why.)

What to submit: Checklist

[ ] Source files for all five tasks, each compiling cleanly with `-fopenmp -Wall`
[ ] Correctness verification code/output included for every parallel version
[ ] Timing tables (thread count vs. execution time) for the specified sizes
[ ] Speedup and/or parallel-efficiency numbers where requested
[ ] Written answers to every "Questions to answer" block above
[ ] In addition, you can place all your code, observations on the git. Submission on GC is compulsory.